

DIGITAL PROGRAMMING REPORT

Hardware Implementation of FP16 Adder and Multiplier Using Verilog HDL

Aasha Chanpa - 202304001

Astha Patel - 202304032

1. Description:-

In this project, an IEEE-754 half-precision (FP16) floating-point adder and multiplier using Verilog HDL on an FPGA platform is designed and implemented. The system classifies each FP16 number (NaN, Infinity, Zero, Subnormal, Normal), supports real-time operand selection, executes floating-point arithmetic, and shows the final 16-bit hexadecimal output on an 8-digit, 7-segment display.

Clean button inputs are guaranteed by a debouncing and one-pulse generation system. Separate modules for top-level operation, display control, addition, multiplication, and classification are integrated into the design. The implementation closely adheres to biasing, exception handling, and IEEE-754 normalisation rules. An FPGA board is used to test the system successfully, and the results for different FP16 inputs are confirmed.

2. Background

Digital signal processing, graphics, machine learning, and embedded computation make wide use of floating-point arithmetic. IEEE-754 defines standardized formats for the representation of real numbers.

The FP16 format has 1 sign bit, 5 exponent bits, and 10 fraction bits (Mantissa) after the leading 1 (implicit).

It is efficient in hardware implementations.

Number Representation

Value = $[(-1)^{\text{sign}}] \times [1.\text{fraction}] \times [2^{(\text{exponent}-15)}]$

1. Zero (Exponent = 0, Fraction = 0)

In IEEE-754 half-precision, zero is the result of the value when the exponent bits are 0 and the fraction bits are also 0. However, the sign bit is still useful because it allows the existence of both +0 and -0. The two zeros act the same in most calculations, but in some functions like signed comparisons they may differ. Zero is considered as an exact value and does not have an implicit leading 1 in the mantissa.

2. Subnormal (Exponent = 0, Fraction != 0)

A subnormal number is identified by zero exponent and a non-zero fraction. Subnormal numbers act as a bridge between zero and the smallest normal floating-point number. In contrast to the normal ones, they do not use the implicit leading 1 in the mantissa; instead, their value is represented with the leading 0. This makes it possible to represent extremely small magnitudes and also to have gradual underflow instead of a sudden change into zero. Subnormals are necessary for keeping precision close to zero, though they are less precise than normal numbers because they effectively have fewer significand bits.

3. Normal ($1 \leq \text{Exponent} \leq 30$)

The concept of normal numbers involves the exponent that is neither all zeros nor all ones. Normal numbers operate the typical single-precision floating-point standard, which assumes an implicit leading 1 in the mantissa. This hidden bit gives additional precision without the need for extra storage in the fraction field. Normal numbers account for the vast majority of real numbers representable in FP16 and allow fast arithmetic operations with stable precision over a large dynamic range.

4. Infinity (Exponent = 31, Fraction = 0)

FP16 denotes infinity as the case when all the bits of the exponent are set to 1 and the fraction bits are all zeros. The sign bit decides if the value is +infinity or -infinity. Infinity is generally the result of overflow, for instance, when the magnitude of the number becomes larger than the

representable range. The operations with infinity follow the rules of mathematics, e.g. adding a finite number to infinity still gives infinity. On the other hand, undefined expressions like infinity - infinity result in NaN.

5. NaN (Exponent = 31, Fraction != 0)

The presence of NaN (Not-a-Number) is indicated by an exponent field of all ones and a fraction field with a non-zero value. NaNs originate from invalid computational operations, for example, $0/0$, $\text{sq_root}[-1]$, or infinity - infinity.

There are two types of NaN distinguished by the most significant bit of the fraction: quiet NaN (qNaN), which goes on silently through most operations and signaling NaN (sNaN), which is designed to cause exceptions or indicate that the operation is undefined.

NaNs are not equal to any other value, including themselves, and their emergence is generally indicative of an error in the calculations.

3. System Architecture/Block Diagram

- Top-Level Blocks:

- 1) Input Section:-

16-bit switches for number inputs.

Buttons debounced for operands selection, operations (ADD/MUL) choosing, and system resetting.

- 2) Arithmetic Units:-

hp_add: Performs FP16 addition with full handling of exceptions.

hp_mul: Performs FP16 multiplication with full handling of exceptions.

- 3) FP16 Classification:-

hp_class helps to determine if a number is zero, subnormal, normal, infinity, or NaN.

- 4) Display Module:-

display32_hex represents the calculated result in hexadecimal on a seven-segment display.

5) Control Logic:-

Controls input latching, result storage, operation selection, and LED display mode.

4. Module Descriptions

1) hp_class (Classifier):-

- Extracts sign, exponent and mantissa
- Identifies NaN, Infinity, Zero, Subnormal, Normal
- Changes the biased exponent to the signed one
- For normalized numbers, a hidden 1 is added
- ADD/MUL as well as LED status indication use it.

2) hp_add (Addition):-

- The smaller significand is shifted according to the exponent difference
- Addition/Subtraction
If signs are the same => add else If signs are different => subtract the larger magnitude from the smaller one.
- Normalization
If overflow => perform shift right 1.
If not then use the leading '1' position
The exponent bias is applied (+15).
- Packing
Sign | exp | mantissa[9:0]

3) hp_mul (Multiplication)

- Multiply the significands ($11 \times 11 \Rightarrow 22$ bits)
- Normalize the product
- Adjust the exponent (add the exponents, subtract the biases)

- Control overflow/underflow
 - Pack FP16 output
- 4) one_pulse & debounce

These modules provide the following guarantees that:-

- There are no bouncing errors of the button
- One and only one clean pulse per press

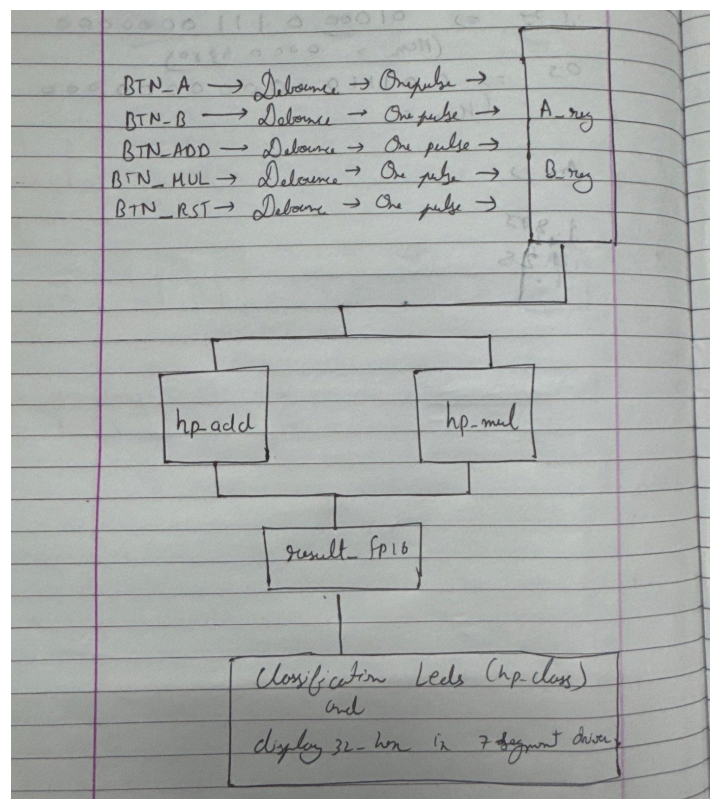
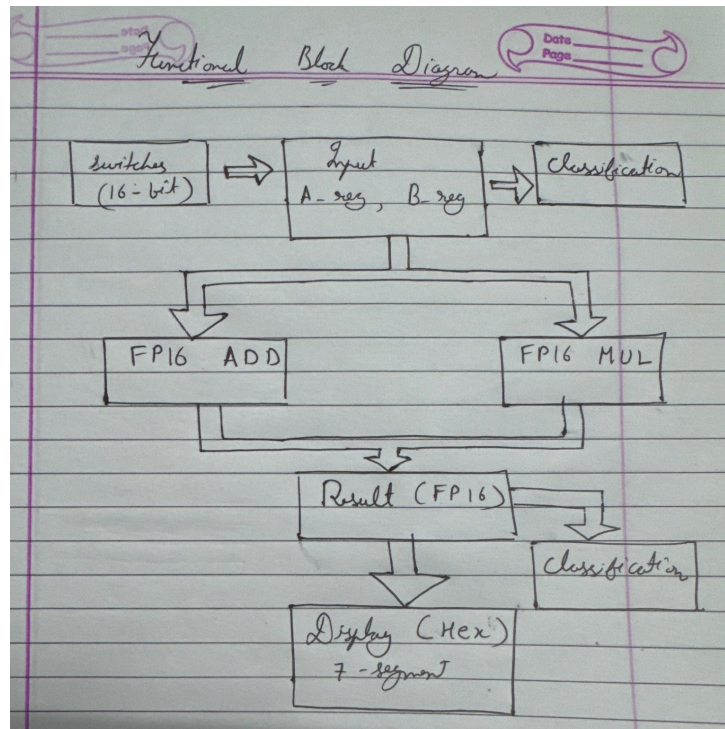
5) display32_hex

- Time-multiplexes 8 digits at the refresh frequency.
- Displays a 32-bit value: the upper 16 bits are zero, the lower 16 bits are the result.

6) top_fp_display

- Controls:
- Button control
- Loading of the registers
- Invocation of hp_add/hp_mul
- LED mode selection
- Result display

5. BLOCK DIAGRAMS



6. Example:-

• MULTIPLY

A = 0.5

0.5 in binary = 0.1

Normalized: 1.0×2^{-1}

- Sign = 0
- Exponent = $-1 + 15 = 14 \Rightarrow 01110$
- Mantissa = 0000000000

So FP16(0.5) = 0 | 01110 | 0000000000 $\Rightarrow 0x3800$

B = 3.75

3.75 in binary = 11.11

Normalize:-

$11.11 = 1.111 \times 2^1$

- Sign = 0
- Exponent = $1 + 15 = 16 \rightarrow 10000$
- Mantissa = 1110000000

So FP16(3.75) = 0 | 10000 | 1110000000 $\Rightarrow 0x4070$

Mathematically:- $0.5 \times 3.75 = 1.875$ and 1.875 in binary = 1.111

Normalize:- 1.111×2^0

Result for 1.875:-

- Sign = 0
- Exponent = $0 + 15 = 15 \Rightarrow 01111$
- Mantissa = bits after the leading 1 = 1110000000 = $0x3F80$

- **ADD :- 0.5 and 3.75**

A = 0.5

B = 3.75

Answer should be 4.25

Binary: 100.01

Normalized: 1.0001×2^2

Exponent = $2 + 15 = 17 \Rightarrow 10001$

Mantissa = 0001000000

NUMBER :- 0100 | 0100 | 0100 | 0000

$\Rightarrow$ Output: 0x4440 (matches hardware)